COMPILER PRINCIPLES LABORATORY REPORT

Experiment 4: Design and Implementation of a Mini Compiler Using Python and PLY

Submitted By

Name: Mostafa Jamal Shakib
Student ID: 202324405018
Department: Computer Science and Technology
University: Nanjing Tech University

1. Purpose and Requirements

The purpose of this laboratory experiment is to understand the fundamental concepts of compiler construction and implement a Mini Compiler using Python and PLY (Python Lex-Yacc). The compiler is designed to process a simplified programming language and perform the major phases of compilation including lexical analysis, syntax analysis, semantic analysis, intermediate code generation, and code optimization.

The main requirements of this experiment are:

- To implement a lexical analyzer that can recognize tokens from source code.
- To implement a syntax analyzer that verifies whether the source code follows the grammar rules.
- To construct an Abstract Syntax Tree (AST).
- To generate and maintain a Symbol Table.
- To generate Intermediate Code using Three Address Code (TAC).
- To perform basic code optimization.
- To provide error detection and reporting mechanisms.
- To develop a graphical user interface for visualizing the compilation process.

The project aims to provide practical knowledge about compiler design and demonstrate how source code is transformed through multiple stages before execution.

2. Grammar

The Mini Compiler supports variable declarations, assignment statements, arithmetic expressions, and special-character-based identifiers.

The grammar implemented in this project is:

program → statement_list

statement_list → statement_list statement

| statement

statement → INT ID ;

| ID = expression ;

expression → expression + expression

| expression - expression

| expression * expression

| expression / expression

| NUMBER

| ID

| (expression)

Examples of valid source code:

```
int a$;

int b#;

int c@;

a$ = 10;

b# = 20;

c@ = a$ + b#;
```

The compiler supports identifiers containing special characters.

Examples:

a$

b#

c@

d&

e*

Identifier Rule:

[a-zA-Z_][a-zA-Z0-9_@$#&*]*

3. Implementation Idea of the Lexical Analysis Program

Lexical Analysis is the first phase of compilation. The lexical analyzer scans the source code character by character and converts the input stream into meaningful tokens.

The lexical analyzer was implemented using Python and PLY Lex.

Tokens Supported

Token Type	Description
INT	Keyword
ID	Identifier
NUMBER	Integer Constant
PLUS	+
MINUS	-
MULT	*
DIV	/
ASSIGN	=
SEMICOLON	;
LPAREN	(
RPAREN	)

Identifier Recognition

The compiler supports identifiers with special characters.

Examples:

a$

b#

c@

d&

e*

Regular Expression:

[a-zA-Z_][a-zA-Z0-9_@$#&*]*

Lexical Analysis Workflow

Source Code

↓

Character Scanning

↓

Token Recognition

↓

Token Generation

↓

Token Stream Output

Example

Input:

int a$;

a$ = 10;

Output:

INT

ID(a$)

SEMICOLON

ID(a$)

ASSIGN

NUMBER(10)

SEMICOLON

The lexer also records line numbers for error reporting and debugging purposes.

4. Implementation Idea of the Syntax Analysis Program

4.1 Elimination of Left Recursion

The grammar contains recursive arithmetic expressions.

Original Grammar:

expression → expression + expression

expression → expression - expression

expression → expression * expression

expression → expression / expression

To avoid ambiguity and infinite recursion, precedence rules were introduced.

precedence = (

('left','PLUS','MINUS'),

('left','MULT','DIV')

)

This ensures proper operator precedence.

4.2 FIRST Sets

FIRST(program)

{ INT, ID }

FIRST(statement)

{ INT, ID }

FIRST(expression)

{ NUMBER, ID, (}

FIRST(term)

{ NUMBER, ID, (}

FIRST(factor)

{ NUMBER, ID, (}

4.3 Parser Design

The parser was implemented using PLY YACC.

The parser supports:

Variable Declaration

int a$;

Assignment Statement

a$ = 10;

Arithmetic Expression

c@ = a$ + b#;

Parenthesized Expression

d& = (a$ + b#) * 2;

After successful parsing, the parser generates an Abstract Syntax Tree (AST).

5. Implementation Idea of Semantic Analysis

5.1 Translation into Intermediate Code

The compiler generates Three Address Code (TAC).

Example 1

Source Code:

c@ = a$ + b#;

Generated TAC:

t1 = a$ + b#

c@ = t1

Example 2

Source Code:

d& = (a$ + b#) * 2;

Generated TAC:

t1 = a$ + b#

t2 = t1 * 2

d& = t2

Quadruple Representation

Operator	Operand 1	Operand 2	Result
+	a$	b#	t1
*	t1	2	t2
=	t2	-	d&

The generated intermediate code simplifies optimization and future code generation.

5.2 Translation into Abstract Syntax Tree

The compiler generates an Abstract Syntax Tree after parsing.

AST Node Types

- ProgramNode
- VarDeclNode
- AssignNode
- VarNode
- ConstNode
- BinOpNode

Example

Source Code:

c@ = a$ + b#;

AST:

AssignNode(c@)

```
    +
  /   \
 a$    b#
```

The AST serves as the central structure used for semantic processing and code generation.

6. Extended Functions and Design Ideas

Several additional features were implemented beyond the basic requirements.

6.1 Special Character Identifier Support

The compiler supports identifiers containing special characters.

Examples:

a$

b#

c@

d&

e*

This feature was implemented by extending the lexical analysis rules.

6.2 Graphical User Interface

A graphical user interface was developed using Tkinter.

The GUI contains the following tabs:

- Source Code
- Tokens
- Parse Trace
- Symbol Table
- Intermediate Code
- Optimized Code
- AST Tree

The interface allows users to observe every compilation phase visually.

6.3 Code Optimization

The compiler implements basic optimization techniques.

Constant Folding

Before Optimization:

t1 = 10 + 20

After Optimization:

t1 = 30

Redundant Assignment Elimination

Before Optimization:

a$ = 0

a$ = 10

After Optimization:

a$ = 10

These optimizations improve code efficiency by reducing unnecessary instructions.

7. Self-Evaluation and Summary

Through this laboratory experiment, I successfully designed and implemented a Mini Compiler using Python and PLY. The project helped me understand the internal architecture of a compiler and the relationship between different compilation phases.

During the implementation process, I developed a lexical analyzer, syntax analyzer, Abstract Syntax Tree generator, Symbol Table generator, Intermediate Code generator, Optimizer, and a graphical user interface. I learned how tokens are recognized, how parsing works, how syntax trees are generated, and how optimization techniques improve program efficiency.

Several challenges were encountered during development, including token recognition conflicts, parser rule design, AST construction, symbol table management, and optimization logic. These challenges were solved through debugging, testing, and gradual refinement of the compiler architecture.

This experiment significantly improved my understanding of compiler principles and provided valuable practical experience in software development. The Mini Compiler successfully performs lexical analysis, syntax analysis, semantic analysis, intermediate code generation, optimization, and visualization of compilation phases.

Overall, this project serves as a strong foundation for future studies in compiler construction, programming language implementation, and advanced software engineering.